AI语音交互(阿里)

AI语音交互(阿里)

- 例程效果
- 代码讲解
  - 1. 导入必要的模块
  - 2. 定义全局变量和常量
  - 3. 辅助函数
    - 3.1 音频上传函数
    - 3.2 录音函数
    - 3.3 WiFi连接函数
    - 3.4 音频API调用函数
    - 3.5 语音合成函数
    - 3.6 音频播放函数
  - 4. 主程序逻辑
  - 5. 异常处理与资源释放

例程效果

本节例程源码文件名为 `audio_chat_tts.py`。

连接IDE，在IDE中运行代码，程序会监听按键：按下按键开始录音，松开按键停止录音，然后将音频上传到云端进行AI分析，获取文本回复，并使用TTS（Text-to-Speech）合成语音，最后播放音频。支持多次对话循环，首次运行会自动连接WiFi，并处理网络错误和资源管理。控制台会实时反馈进度和状态。

> 注意：
>
> 1. 本节代码的输出全部在控制台进行，屏幕上不会有任何显示内容
> 2. 音频输出需要配备扬声器

```
[DEBUG] Response status code: 200
[DEBUG] Total response size: 869 bytes
[DEBUG] Response status code: 200
[DEBUG] Closing connection
响应状态码: 200, b'{"output":{"choices":[{"finish_reason":"stop","message":{"content":[{"text":"\xe4\xbd\xa0\xe5\xa5
音频API请求成功!
JSON解析成功
提取到AI回复文本: 你好，录音测试正常。
AI回复: 你好，录音测试正常。

正在合成语音...
开始语音合成: 你好，录音测试正常。...
发送TTS请求...

[DEBUG] Starting request: POST https://dashscope.aliyuncs.com/api/v1/services/aigc/multimodal-generation/generatio
[DEBUG] Parsed URL: {'path': '/api/v1/services/aigc/multimodal-generation/generation', 'query': '', 'scheme': 'htt
[DEBUG] Custom headers: {'Authorization': 'Bearer sk-75663a0b07eb45049e715c850c5cffed', 'Content-Type': 'applicat
[DEBUG] Final headers: {'Connection': 'close', 'User-Agent': 'MicroPython/1.0', 'Host': 'dashscope.aliyuncs.com',
[DEBUG] Body length: 128
[DEBUG] Full path: /api/v1/services/aigc/multimodal-generation/generation
[DEBUG] Request data length: 262
[DEBUG] Creating socket connection to dashscope.aliyuncs.com:443
[DEBUG] Resolved address: ('47.93.243.29', 443)
```

代码讲解

1. 导入必要的模块

```
import uos
import time
import gc
import ujson as json
from media.display import *
from media.media import *
from media.pyaudio import *
import media.wave as wave
from ybUtils.YbSpeaker import YbSpeaker
from ybUtils.YbKey import YbKey
import YbRequests as requests
import libs.upload_image as upload_image
```

- `uos`：用于操作系统相关操作，如文件删除和退出点管理。代码中用于删除临时文件和退出程序。
- `time`：用于时间控制，包括延时（如 `time.sleep_ms()`）和获取时间戳（如在文件名中）。
- `gc`：用于垃圾回收，确保程序在处理大文件（如音频）时不会内存溢出。代码中在关键点（如上传前后）显式调用 `gc.collect()`。
- `ujson`：用于JSON数据处理，支持解析API响应。
- `media.display`，`media.media`，`media.pyaudio`，`media.wave`：这些是媒体处理模块，用于屏幕显示、媒体管理、音频录制和WAV文件操作。`pyaudio` 处理音频流，`wave` 用于读写WAV文件。
- `ybUtils.YbSpeaker`：扬声器控制模块，用于音频播放。
- `ybUtils.YbKey`：按键检测模块，用于监听用户输入以启动录音。
- `YbRequests`：自定义HTTP请求库，用于发送API请求，如音频分析和TTS合成。
- `libs.upload_image`：自定义模块，用于上传文件到OSS（对象存储服务），尽管是图像库，但被重用于音频上传，体现了代码的复用性。

这些导入确保了程序的音频输入/输出、网络通信和UI显示能力，是构建音频对话系统的核心依赖。

2. 定义全局变量和常量

```
API_KEY = ""  # 请替换为您的API密钥
WIFI_SSID = ""
WIFI_KEY = ""
TTS_OUTPUT_FILE = "/data/tts_output.wav"
RECORDED_AUDIO_FILE = f"/data/recorded_audio_{time.ticks_ms()}.wav"
spk_global = YbSpeaker()
media_manager_initialized = False
```

- `API_KEY`，`WIFI_SSID`，`WIFI_KEY`：这些是用户配置的常量，用于API认证和WiFi连接。API_KEY是DashScope服务的密钥，需替换为实际值。
- `TTS_OUTPUT_FILE`：指定TTS合成的音频输出路径（"/data/tts_output.wav"），用于保存合成的语音文件。
- `RECORDED_AUDIO_FILE`：动态生成录音文件的路径，使用 `time.ticks_ms()` 确保每次录音文件名唯一，避免冲突。
- `spk_global = YbSpeaker()`：全局扬声器对象，用于音频播放。
- `media_manager_initialized`：布尔标志，用于确保MediaManager只初始化一次，优化资源使用。

这些变量定义了程序的配置和状态，体现了代码的模块化和可维护性。

3. 辅助函数

代码中包含多个辅助函数，每个函数负责特定功能，并包含错误处理和资源管理。

3.1 音频上传函数

```
def upload_audio_to_oss(audio_path, max_retries=5):
    """使用稳定的upload_image库上传音频到OSS，支持重试机制"""
    print("开始上传音频到OSS...")

    gc.collect()  # 垃圾回收前进行内存清理

    for attempt in range(1, max_retries + 1):
        try:
            progress = attempt * 20  # 模拟进度
            print(f"上传中({progress}%)")

            oss_url = upload_image.upload_image_to_dashscope(API_KEY,
audio_path, "qwen-audio-turbo")

            print("上传中(100%)")
            print(f"音频上传成功! OSS地址: {oss_url}")

            gc.collect()  # 上传成功后再次垃圾回收
            return oss_url
        except Exception as e:
            gc.collect()  # 异常时垃圾回收
            if attempt < max_retries:
                wait_time = 0.1  # 等待时间固定为0.1秒
                time.sleep(wait_time)
            else:
                print("上传失败，请检查网络连接")
                raise e
```

- **功能**：上传音频文件到OSS，使用 `upload_image.upload_image_to_dashscope` 函数（这里命名虽然是图像库，但是被复用于音频了）。这是一个重试机制函数，支持最多5次尝试。
- **详细逻辑**：函数先进行垃圾回收以释放内存，然后循环尝试上传。每次尝试前打印进度（如20%、40%），上传成功后返回OSS URL。异常时立即垃圾回收，并在重试间添加延迟。
- **错误处理**：捕获所有异常，并在最大重试次数后抛出错误，允许调用者处理。

3.2 录音函数

```
def record_with_button_and_upload(filename, key, left_volume=85,
right_volume=85):
    """按键录音并上传，返回OSS链接"""
    global media_manager_initialized

    print("按下按键开始录音，松开按键停止录音...")

    while not key.is_pressed():
        time.sleep_ms(10)

    FORMAT = paInt16
    CHANNELS = 1
    RATE = 44100
    CHUNK = RATE // 25
```

```
    frames = []

    p = PyAudio()
    p.initialize(CHUNK)

    if not media_manager_initialized:
        MediaManager.init()
        media_manager_initialized = True

    input_stream = p.open(format=FORMAT, channels=CHANNELS, rate=RATE,
input=True, frames_per_buffer=CHUNK)
    input_stream.volume(LEFT, left_volume)
    input_stream.volume(RIGHT, right_volume)

    print("开始录音...")
    while key.is_pressed():
        try:
            data = input_stream.read(CHUNK)
            frames.append(data)
        except Exception as e:
            print(f"录音帧读取错误: {e}")
            break
        time.sleep_ms(10)

    print("停止录音...")
    input_stream.stop_stream()
    input_stream.close()

    gc.collect()  # 录音后垃圾回收

    try:
        wf = wave.open(filename, 'wb')
        wf.set_channels(CHANNELS)
        wf.set_sampwidth(p.get_sample_size(FORMAT))
        wf.set_framerate(RATE)
        wf.write_frames(b''.join(frames))
        wf.close()
        print("录音文件保存完成")

        gc.collect()  # 保存后垃圾回收
    except Exception as e:
        print(f"保存录音失败: {e}")
        return None

    try:
        oss_url = upload_audio_to_oss(filename)
        return oss_url
    except Exception as e:
        print(f"上传录音失败: {e}")
        return None
```

- **功能**：监听按键进行录音，保存为WAV文件，然后上传到OSS，返回上传链接。
- **详细逻辑**：函数先等待按键按下，然后初始化音频流并录制数据直到按键松开。录制后保存文件并上传，支持音量调整。使用全局标志确保MediaManager只初始化一次。
- **错误处理**：在录制循环中捕获异常，录音后立即垃圾回收。上传失败时返回None。

3.3 WiFi连接函数

```
def connect_wifi():
    """连接WiFi网络"""
    print("正在连接WiFi...")
    try:
        import network
        wlan = network.WLAN(network.STA_IF)
        wlan.active(True)

        if not wlan.isconnected():
            wlan.connect(WIFI_SSID, WIFI_KEY)
            timeout = 15
            while not wlan.isconnected() and timeout > 0:
                time.sleep(1)
                timeout -= 1
                print(f"连接中... 剩余时间: {timeout}s")

            if wlan.isconnected():
                print(f"WiFi连接成功! IP: {wlan.ifconfig()[0]}")
                return True
            else:
                print("WiFi连接超时")
                return False
        else:
            print("WiFi已连接")
            return True
    except Exception as e:
        print(f"WiFi连接失败: {e}")
        return False
```

- **功能**：连接WiFi网络，支持超时等待。
- **详细逻辑**：激活WLAN接口，尝试连接指定SSID和KEY，循环检查连接状态直到超时。成功返回True，否则False。
- **错误处理**：捕获所有异常，确保函数鲁棒性。

3.4 音频API调用函数

```
def call_audio_api(oss_url):
    """调用音频API获取AI回复"""
    print("开始调用音频API...")

    gc.collect()  # API调用前垃圾回收

    url = "https://dashscope.aliyuncs.com/api/v1/services/aigc/multimodal-
generation/generation"
    headers = {
        "Authorization": f"Bearer {API_KEY}",
        "Content-Type": "application/json",
        "X-DashScope-OssResourceResolve": "enable"
    }

    data = {
        "model": "qwen-audio-turbo-latest",
        "input": {
            "messages": [
```

```
                {
                    "role": "system",
                    "content": [{"text": "You are a helpful assistant."}]
                },
                {
                    "role": "user",
                    "content": [{"audio": oss_url}]
                }
            ]
        }
    }

    try:
        response = requests.post(url, headers=headers, json_data=data,
timeout=60)

        if response.status_code == 200:
            try:
                response_text = response.text
                if isinstance(response_text, bytes):
                    response_text = response_text.decode('utf-8')
                result = json.loads(response_text)

                if 'output' in result and 'choices' in result['output']:
                    choices = result['output']['choices']
                    if len(choices) > 0 and 'message' in choices[0]:
                        content_list = choices[0]['message']['content']
                        if len(content_list) > 0 and 'text' in content_list[0]:
                            ai_text = content_list[0]['text']
                            gc.collect()  # API成功后垃圾回收
                            return True, ai_text
            except Exception as json_error:
                gc.collect()
                return False, f"JSON解析失败: {json_error}"
        else:
            gc.collect()
            return False, f"API请求失败: {response.status_code}"
    except Exception as e:
        gc.collect()
        return False, f"请求异常: {e}"
```

- **功能**：使用上传的音频URL调用AI API获取文本回复。
- **详细逻辑**：构建请求数据，包括系统提示和用户音频，发送POST请求。解析响应提取文本，处理各种错误。
- **错误处理**：捕获网络和解析异常，并在每个关键点进行垃圾回收。

3.5 语音合成函数

```
def synthesize_speech(text, voice="Cherry"):
    """调用DashScope TTS API进行语音合成"""
    print(f"开始语音合成: {text[:50]}...")

    try:
        url = "https://dashscope.aliyuncs.com/api/v1/services/aigc/multimodal-
generation/generation"
        headers = {"Authorization": f"Bearer {API_KEY}", "Content-Type":
"application/json"}
```

```
        data = {"model": "qwen3-tts-flash", "input": {"text": text, "voice":
voice, "language_type": "Chinese"}}

        response = requests.post(url, headers=headers, json_data=data,
timeout=60)

        if response.status_code == 200:
            try:
                response_text = response.text
                if isinstance(response_text, bytes):
                    response_text = response_text.decode('utf-8')
                result = json.loads(response_text)

                if 'output' in result and 'audio' in result['output'] and 'url'
in result['output']['audio']:
                    audio_url = result['output']['audio']['url']
                    audio_response = requests.get(audio_url, timeout=30)

                    if audio_response.status_code == 200:
                        with open(TTS_OUTPUT_FILE, 'wb') as f:
                            f.write(audio_response.content)
                        print(f"音频文件保存成功: {TTS_OUTPUT_FILE}")
                        return True, "语音合成成功"
            except Exception as json_error:
                return False, f"JSON解析失败: {json_error}"
        else:
            return False, f"API请求失败: {response.status_code}"
    except Exception as e:
        return False, f"语音合成失败: {e}"
```

- **功能**：将文本转换为语音音频文件。
- **详细逻辑**：发送TTS请求，下载音频URL并保存为WAV文件。
- **错误处理**：全面捕获异常。

3.6 音频播放函数

```
def play_audio():
    """播放合成的音频"""
    try:
        with open(TTS_OUTPUT_FILE, 'rb') as f:
            pass
    except OSError:
        return False, "音频文件不存在"

    print(f"开始播放音频: {TTS_OUTPUT_FILE}")

    try:
        spk_global.enable()
        wf = wave.open(TTS_OUTPUT_FILE, 'rb')
        CHUNK = int(wf.get_framerate()/25)
        p = PyAudio()
        p.initialize(CHUNK)
        stream = p.open(format=p.get_format_from_width(wf.get_sampwidth()),
channels=wf.get_channels(), rate=wf.get_framerate(), output=True,
frames_per_buffer=CHUNK)
        stream.volume(vol=100)
        data = wf.read_frames(CHUNK)
```

```
        while data:
            stream.write(data)
            data = wf.read_frames(CHUNK)
    except BaseException as e:
        return False, f"播放失败: {e}"
    finally:
        stream.stop_stream()
        stream.close()
        p.terminate()
        wf.close()
        spk_global.disable()

    return True, "播放完成"
```

- **功能**：播放保存在文件中的音频。
- **详细逻辑**：打开音频流，读取并写入数据。
- **错误处理**：Finally块确保资源释放。

4. 主程序逻辑

```
def main():
    """主函数 - 持续的音频对话+语音合成循环"""
    print("K230音频对话+语音合成整合程序")

    try:
        if not connect_wifi():
            return

        key = YbKey()
        conversation_count = 0

        while True:
            conversation_count += 1
            current_audio_file = f"/data/recorded_audio_{time.ticks_ms()}.wav"
            oss_url = record_with_button_and_upload(current_audio_file, key)

            if oss_url:
                success, ai_text = call_audio_api(oss_url)
                if success:
                    success, message = synthesize_speech(ai_text)
                    if success:
                        success, message = play_audio()

            time.sleep(1)
    except Exception as e:
        print(f"程序异常: {e}")
```

- **功能**：主循环处理连续对话。
- **详细逻辑**：连接WiFi，循环录音、分析、合成和播放。
- **错误处理**：异常时跳过对话。

5. 异常处理与资源释放

- 代码在函数和主程序中使用try-except-finally块，确保异常时资源释放，如音频流和扬声器。